

Spot The Ball

How the benchmark works, why every design choice is what it is, and what to read first.

Onboarding document · `spot-the-ball-2.0/` · September 2026

Companion paper: Balamurugan et al., "Spot The Ball: A Benchmark for Visual Social Inference", arXiv:2511.00261

1. What we are actually trying to find out

Watch three seconds of soccer. Without any effort you know who is on which team, who is about to receive the pass, which player is out of position, and — even if the ball goes behind a cluster of bodies — roughly where it must be. Your brain builds a small model of the situation and runs it forward.

Vision-language models (the image-and-text systems behind ChatGPT, Gemini, Claude) are good at *describing* a soccer field. They are much weaker at that inferential layer. This project builds a benchmark that measures precisely where the gap is, by asking humans and models the same questions about the same clips.

The flagship task, in one sentence. Show a short clip in which the ball becomes invisible partway through, then ask where the ball ended up. You cannot answer by looking for the ball. You can only answer by reading the players — where they are facing, who is converging, who just struck it.

That task is the whole reason for every strange-sounding decision in this repo. If a clip lets you find the answer any other way, the clip is broken.

2. Why a simulator instead of real footage

We generate every clip in **gfootball** (Google Research Football), a soccer simulator Google released in 2019 to train soccer-playing agents. We are not training anything. We use it as a *controlled video generator*.

Real broadcast footage is unusable for this, for four reasons:

- **It leaks answers.** Score graphics, commentary, replay captions.
- **It is uncontrolled.** Camera cuts, varying weather, varying lighting.
- **Players are identifiable.** Numbers, hairstyles, body types, boots — a model could recognise a player rather than reason about a role.
- **There is no ground truth.** Nobody can tell you the exact pixel the ball occupied on frame 97.

The simulator fixes all four. Identical lighting every time, one camera, anonymous players, and — because we control the world — the exact answer to every question we ask.

The trade we accept. Simulated football is not real football. It can look robotic, and passages of play can look nonsensical if you set it up carelessly. A large part of this project's engineering exists to *measure* whether a given clip looks like real football, and to throw away the ones that do not. See §8.

3. The answer grid, and the four versions we tried

If we asked "where is the ball?" and accepted free text, we could not grade the answers. So we overlay a labelled grid on the video and the answer becomes a cell name, like `D15` . Rows are letters top to bottom, columns are numbers left to right.

Every clip is rendered at **1280 × 480 pixels** — a wide crop of the playing field. The grid has to tile that rectangle exactly, in *square* cells, or some cells would be easier to hit than others.

How the grid evolved

VERSION	CELLS	CELL SIZE	WHAT HAPPENED
6 × 4	24	n/a	The first attempt. The grid was <i>described in the text prompt only and never drawn on the image</i> . Abandoned — a model and a human were effectively imagining two different grids, and neither matched ours.
6 × 10	60	rectangular	What the reference paper uses. We started from it, but wanted finer resolution and square cells.
32 × 12	384	40 × 40 px	Burned onto every frame of the video, deliberately much finer than the paper. Used for the early batches. Too fine in practice — see below.
16 × 6	96	80 × 80 px	The locked format. Rows A–F, columns 1–16, drawn in yellow.

On **2026-07-08** we rendered the same two clips with four labelled grids burned in side by side — `32×12` , `16×6` , `28×8` , `8×3` — and compared them directly. That comparison is what settled it.

16 × 6 — the locked grid. 1280 / 16 = 80 px. 480 / 6 = 80 px. Square, and it tiles the frame exactly.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A																
B																
C																
D															**	
E																
F																

The `**` marks cell **D15** — row D, column 15. That string is a complete, gradable answer.

Why 16 × 6 and not the finer grid

- **384 cells is finer than anyone's actual certainty.** At 40 px a cell, neither a human nor a model can meaningfully distinguish a cell from its neighbour on a moving 1280-wide video. You get precision you cannot justify.
- **It makes the scoring dishonest.** On a 384-cell grid, "one cell off" and "the other end of the pitch" both score zero. The grid should be coarse enough that landing in the right cell means you were genuinely right.

- **It makes the statistics unusable.** A batch is a few dozen clips. Spread across 384 cells, almost every cell is empty and you can say nothing about coverage. Across 96 you can: GEN3_HARD's 24 answers land in 23 distinct cells, covering all 6 rows and 14 of 16 columns.
- **96 cells is labelable.** A–F and 1–16 is easy to say out loud, easy to type into a form, and impossible to misread.

Locked. The benchmark is fixed at 16×6 with no player names. Do not revert to 32×12 , and do not render names. Anything produced in another format cannot be pooled with the existing batches.

4. What one clip looks like

Every clip in the current format:

PROPERTY	VALUE
Resolution	1280×480 (wide crop of the pitch)
Duration	5.0 seconds (the intro clips are 20 s)
Frame rate	25 fps $\rightarrow$ 125 frames
Grid	16×6 , yellow, rows A–F, columns 1–16, burned into every frame
Team A kit	Blue outfield, pale sky blue goalkeeper
Team B kit	Red outfield, pale coral goalkeeper
Officials	Invisible (rendered at 2% scale)
Player names	None — rendered with a blank font
Start marker	Red circle around the ball on the first frame only

Each clip ships in **two versions**:

- `full_visibility/` — ball visible for all 5 s. Used for checking and for tasks that are not the hidden-ball task.
- `split_1s_4s/` — ball visible for 1 second, then invisible for 4 seconds. **This is the actual stimulus.** The subject must name the cell the ball is in on the final frame.

What "invisible" means here — this trips everyone up. The ball is *not* removed from the simulation. It is shrunk until it cannot be seen. It is still there, still being passed, still governing where every player runs. Same for the referee, and same for the hidden-player experiments. If we deleted these objects the football would change and the question would have no answer.

Why goalkeepers wear a pale version of their own team's colour

They used to wear yellow — a colour belonging to neither side. Viewers read a yellow keeper as a *third team*, which is exactly the wrong inference. Pale blue and pale coral keep each keeper legibly part of his own side while still marking him as the keeper.

5. Why we never show a goal

This is one of the firmest rules in the project, and it has three separate reasons.

1. **A goal teleports the ball.** When a goal is scored the engine instantly respots the ball on the centre mark — measured as a 1.017-unit jump in a single frame. If that happens inside a clip, the ball's path is discontinuous and the question "where did it end up?" has no sensible answer.
2. **A goal resets both teams.** All 22 players snap back to their formation. The passage of play you were watching simply stops existing.
3. **A goal is a different, easier question.** Once the ball is in the net, everyone's body language tells you where it went. The task becomes reading celebration rather than reading play.

So every clip is **one unbroken passage of continuous play**. We enforce this differently depending on the batch:

- **In the drills** (§6), the renderer detects any goal, goal-line crossing, or teleport, and discards that seed entirely, keeping only seeds where the ball keeps circulating.
- **In the match batches** (§7), the match is configured with `end_episode_on_score = False`, so play continues after a goal and we get more usable footage per match. A window may then *begin* at a post-goal restart — but the window is forced to start at the restart frame itself, so the goal and the respot are never inside the clip.

Related: `end_episode_on_out_of_play` and `end_episode_on_possession_change` are also off, for the same reason — we want long continuous matches to cut windows from, not short fragments.

6. The drills era — hand-built scenarios

The first serious clip batches were **drills**: small, hand-authored set-ups where we placed players ourselves and let the engine play it out. The point was coverage — each drill produces a characteristically different kind of ball movement, so the benchmark is not 40 clips of the same thing.

There are eight, in `DRILLS/` (70 clips rendered):

#	DRILL	WHAT KIND OF BALL MOVEMENT IT PRODUCES
1	4v4 small-sided	Balanced open play through the middle. The baseline.
2	3v1 rondo (keep-away)	Tight, fast handoffs in a small space. Short total ball path (~0.3–0.4).
3	5v2 rondo	Wider circulation with passes split through the middle.
4	Fast break 3v2	Directional forward travel — the ball covers ground toward goal.
5	Wing cross	Long sideways travel across the final third.
6	Switch of play	Maximum lateral displacement — ball crosses the full width (~1.0).
7	Free kick	Set piece: dead ball outside the box worked back into open play.
8	Corner kick	Set piece: delivery from the corner flag circulated in the box.

Each drill runs in **3 position variants** (small jitters to the starting layout) × **2 versions** (visible / split), which is where the clip count comes from. Set-piece variants with no goal-free seed are skipped rather than shipped.

The one drill we had to fix

Wing cross originally kept producing goals, because a cross into the goalmouth is a shot waiting to happen. It was re-aimed to circulate the ball *across and away from* the goalmouth, so the passage stays alive. This is the rule in §5 biting in practice.

7. Why we moved on from drills, and what replaced them

Drills gave us control, but control turned out to be the problem. A hand-placed scenario is a *staged* situation: we decided in advance that this would be a rondo, so the clip shows a rondo. Two consequences:

- **The football is only as good as our guess.** We are not soccer coaches. Hand-placed positions produce play that a real analyst would find slightly off.
- **It does not scale, and it is not natural.** Eight drills is eight shapes of play, forever. Real matches produce situations we would never think to stage.

So the approach inverted. Instead of *staging* a situation, we now **run full 11-a-side matches and go looking for situations that occurred naturally.**

The pipeline, in five steps.

1. **SWEEP** — play many full matches, varying the starting shape and the random seed, and log every frame of every match.
2. **DETECT** — scan those logs for the situation types we want (a corner, a keeper's throw, a kick-off, a stretch of open play).
3. **SCORE** — measure each candidate 5-second window against the coherence statistics in §8, and reject the ones that do not read as football.
4. **SELECT** — pick the surviving windows that give the batch the spread it needs (headcounts, situation mix, ball-position spread, team balance).
5. **RENDER** — render each chosen window twice, visible and split, and measure the ground truth.

The starting shapes are still ours — there are **14** of them, and they steer *where on the pitch play tends to happen*, which in turn controls how many bodies the camera frames:

SHAPE FAMILY	WHAT IT SETS UP	HEADCOUNT IT PRODUCES
Midfield (5 shapes)	Both banks of players in shot	High — 16 to 19 people
Flank / box (4 shapes)	Pressure in the final third	Source of corners and keeper throws
Wide + even (2 shapes)	Contested wide play that stays alive	Middle — 11 to 15
Counterattack (2 shapes)	One side committed high, the other deep	Low — 8 to 10

The counterattack shapes matter more than they look. With a ball-tracking camera, "few players on screen" almost always means play is jammed into a corner and about to go out of bounds — so low headcounts and stoppages become the same event, which would confound the benchmark. A genuine breakaway is the one situation where live, attended play really does frame only a handful of bodies.

The four situation classes we settled on

From everything the drills explored, the batches narrowed to four detectable classes. A 24-clip batch is always split the same way:

CLASS	COUNT	WHY IT IS IN THE SET
Open play	9	The default. Continuous possession and passing with no restart involved.
Corner kick	5	Many bodies packed into the box — the hardest crowding case.
Goalkeeper distribution	5	Ball travels a long way from a known origin. Tests long-range inference.
Kick-off	5	Both teams in known formation. The cleanest, most legible starting state.

A real engine quirk worth knowing. gfootball never labels a kick-off. Measured across 120 cached matches — 330,000 frames — the kick-off game mode fired *exactly zero times*, while goal kicks and free kicks fired constantly. Restarts still happen; the engine just does not tag them. So we detect kick-offs by *behaviour* instead: the ball respotted on the centre mark with both sides behind the halfway line. Corners are similarly rare — roughly 0.03 per match — which is why corner clips are always the scarcest and the slowest to collect.

8. Coherence — how we decide a clip looks like football

This is the most important quality mechanism in the project, and it exists because of a failure.

One batch (GEN2 , second version) produced clips that were obviously wrong: bouncy, illogical, not football. The intuitive suspect was the physics engine. We measured it, and physics was **ruled out**. The real causes were mundane configuration: lopsided team difficulty, offsides switched off, and a bad selection gate.

The fix was to stop eyeballing clips and start measuring them. Every candidate window gets four numbers:

loose

Fraction of frames where *nobody* has possession. High means the ball is pinballing around rather than being played.

apex

The maximum height the ball reaches. High means it spent the clip in the air rather than being worked along the ground.

dnear

Median distance from the ball to the nearest player. High means the ball is out in space with nobody involved.

path

Total distance the ball travelled. Gated at both ends — too low is a static clip, too high is frantic.

These combine into a single score:

$$\text{coherence} = \text{loose} \times 2.0 + \text{apex} / 3.2 + \text{dnear} / 4.0$$

LOWER IS BETTER. A low score means a mostly-owned ball, kept low, kept close to a player. This is genuinely easy to get backwards, and it has already been gotten backwards once in a RESUME file — see §12.

The thresholds are **per situation class**, not global. A corner is an unowned ball in flight *by definition*, and a keeper's delivery goes up in the air on purpose. Holding those to open play's numbers would reject every single one. Each class gets the bounds its own physics allows, and within a class the ranking picks the calmest instance.

9. How we know the right answer

We never estimate the ground truth from the video. We **measure** it, using a technique called *hide-and-diff* that appears everywhere in this project.

Ball position

Render the exact same deterministic passage of play twice — once with the ball visible, once with it invisible — and subtract the two image sequences. Everything identical cancels out. What remains is the ball, and its pixel position is exact. That pixel is then mapped to a grid cell.

Counting people in frame

Same idea, 23 times. Render a "plate" with all 22 players hidden, then render 22 more passes, each showing exactly one player, and diff each against the plate. A player whose diff has enough changed pixels is on screen. This gives an exact per-player, per-frame answer.

Why not just count coloured blobs? Because it does not work. The advertising hoardings around the pitch are saturated blue and red — the same colours as the kits — which biases a blob count by about **+6 people**. Fitting the camera frustum geometrically was also tried, and gets the per-frame count exactly right only 36% of the time. Plate-versus-solo is the only exact method.

The cost is real: 23 render passes per clip, and every pass has to replay the match from frame 0 to reach the window. For a 24-clip batch that is about 50 minutes on three parallel workers. Budget for it — it is not optional if you need per-frame counts.

10. The engine modifications

Stock gfootball cannot produce these clips, so we maintain four small C++ patches in `engine_patches/gfootball_engine.patch`. Each is driven by an environment variable and is a **no-op when unset**, so an unpatched-looking run is byte-identical to stock.

PATCH	SWITCH	WHAT IT DOES
<code>match.cpp</code>	<code>GFOOTBALL_CAM_OFFSET_X/_Y</code>	Shifts the camera's aim point off the ball.
<code>humanoidbase</code>	(mechanism only)	Adds <code>renderScale</code> — shrinks a player's <i>drawn</i> body.
<code>playerofficial.cpp</code>	always on	Officials at 2% scale — invisible referees, in every clip.
<code>team.cpp</code>	<code>GFOOTBALL_HIDE_SLOTS</code> <code>GFOOTBALL_TEAM_GK_KITS</code>	Hides named players while they stay in play; gives each keeper his own team's colour.

Why the camera patch exists

Stock gfootball aims the camera *at the ball*. The ball therefore renders dead centre in every single frame, and every grid answer collapses into the middle few cells — which destroys the benchmark. The offset shifts the aim point so the ball sits off-centre, the way a real camera operator frames play. This is why GEN3_HARD's answers spread across 23 of 96 cells instead of clustering.

None of these patches touch physics. Ball dynamics live in `ball.cpp` as compile-time constants (bounce 0.62, drag 0.015, friction 0.04, gravity -9.81) and read nothing from players. The camera patch modifies a local copy of a predicted ball position used only as a look-at target — it cannot move the ball. The shrink patches touch the render mesh only, leaving the source mesh full-size. This separation is load-bearing for the whole methodology: if hiding a player changed the football, every hide-and-diff measurement in §9 would be meaningless.

11. Difficulty — the one dial that changes the football

gfootball exposes a single **difficulty** value per team, 0.0 to 1.0, frozen at kickoff. It is read in exactly three places in the entire engine: a multiplier on all player stats, added reaction latency, and how far a player will chase an opponent.

It is worth knowing two things about it:

- **It is per team, not per player.** All 11 players share one number. There is no per-player setting and no way to add one from the Python side — it would take a small engine patch (which would not require touching physics).
- **Both squads are otherwise identical.** The engine hardcodes 11 player profiles with differing stats, and assigns the *same 11* to both teams. So difficulty is the only asymmetry lever available.

That last point explains a real trade-off in the current batches. `GEN3` ran its final-third shapes lopsided — one side at 1.00 against 0.60 — specifically to manufacture attacking pressure, which is where corners and keeper throws came from. `GEN3_HARD` runs both sides at 0.95 (gfootball's own "hard" preset), which removes that lever entirely.

12. What exists right now

BATCH	CLIPS	STATUS	WHAT IT IS
DRILLS	70 files	Built	The 8 hand-authored drills.
MATCH_SITUATIONS	22 files	Built	Set-piece and player-count drills.
30_1s_visible_4s_invisible	30 files	Built	The first batch in the 1s/4s split format. Has a Gemini evaluation attached.
GEN2	122 files	Paused	A 160-clip classification set, stopped mid-build. See <code>GEN2/RESUME.md</code> .
GEN3	24 clips	Built & verified	The pilot. Exact headcounts 8–19, two per level.
GEN3_HARD	24 clips	Built & verified	The same spec at hard difficulty both sides.
INTRO	2 clips	Built	20-second clips for the experiment's instructions page.

Beyond the video, the built batches also ship:

- **ground_truth.csv** — one row per clip: situation, headcount, ball start and final cell, exact pixels, the seed and frame window, and the coherence numbers.
- **scene_graphs/** — per-frame JSON of the full game state: every player's position, velocity and role, possession, score, game mode.
- **Exact reproducibility** — `reproduce_clip.py` rebuilds any clip as a byte-identical file. One known limit: a counterfactual must branch at frame 0, because the engine's state-setting cannot intervene mid-clip.

Open correction, not yet applied. GEN3_HARD/RESUME.md states that hard-versus-hard play scored "materially more coherent". That reads the scale backwards. Recomputed from the shipped CSVs: GEN3 = **0.743**, GEN3_HARD = **0.844**, and lower is better — so the hard batch is *less* coherent, driven by a looser ball (0.165 vs 0.127) and more air time (1.112 vs 0.998). The figure quoted in the file (0.858) also does not match the shipped data (0.844). Fix this before anyone cites it.

13. Where to start

1. **Watch clips before reading code.** Open GEN3/clips/full_visibility/clip_01.mov, then the matching split_1s_4s/ version, and try the task yourself. Nothing else in this document will land until you have.
2. **Open GEN3/clips/ground_truth.csv** next to the clips you just watched. Every column is explained in §12.
3. **Read docs/README.md** — the original orientation doc.
4. **Read GEN3/RESUME.md** — the build log for the pilot, including what went wrong and how many audit rounds it took.
5. **Then the code**, in this order: gen3_lib.py (shapes, coherence, situation detectors), gen3.py (the phases), and gen3_report.py (the review charts).

Vocabulary you will hit immediately

TERM	MEANING
shape	A starting formation + ball position + difficulty setting. There are 14.
seed	The random seed for a match. Shape + seed identifies a specific match.
window	The 125-frame stretch of a match that becomes a clip.
slot	A player's index within a team, written L0 – L10 / R0 – R10.
plate	A render with everything hidden, used as the baseline for diffing.
split	The 1s-visible / 4s-invisible version — the real stimulus.
bundle	A packaged set of game assets (kits, fonts, ball size) the engine loads at startup.
sweep	The phase that plays many matches and logs them.
probe / trace	The 23-pass hide-and-diff measurement of who is on screen.

Two habits that will save you. First: the engine leaks memory and a process dies at roughly its 46th match with no error message — this is why every phase does a bounded amount of work, exits, and is restarted by a shell loop. That is not a bug, it is the workaround. Second: exit codes are how this pipeline reports success. Never put a filter or a pipe between a stage and its status — it has already silently swallowed a failure once.